



Факультет экономических наук

Базовая кафедра инфраструктуры
финансовых рынков

Москва

Библиотека Requests

Формируем HTTP-запросы и получаем данные веб-страниц на Python

Бакланова Валерия Сергеевна

преподаватель БКИФР, младший научный сотрудник ЦФИАНД



HTTP-запросы — фундамент интернета: каждый раз, открывая страницу, браузер запрашивает у сервера данные, а сервер отвечает всем необходимым для её отображения.

Requests значительно упрощает формирование таких запросов на Python — сложно представить разработку парсера без неё.

В этом разделе мы научимся:

- формировать запросы разными HTTP-методами;
- настраивать заголовки, User-Agent и прокси;
- анализировать данные запросов и ответов;
- снижать риск блокировки при парсинге сайтов.

```
import requests

response = requests.get('https://www.example.com')
print(response.text)
```



- **Простота и интуитивность** — даже сложные запросы пишутся в 2–3 строки понятного кода.
- **Функциональность** — поддержка всех HTTP-методов, куки, сессий, заголовков — почти всё нужное для веб-скрейпинга и работы с API.
- **Встроенная аутентификация** — готовые механизмы для базовой и более сложной авторизации.
- **Документация и сообщество** — огромное количество примеров и готовых решений.
- **Переиспользуемость** — высокий уровень абстракции позволяет легко переносить код между проектами.



- **Не создана для асинхронности** — библиотека синхронная; для асинхронных запросов используют другие инструменты, например aiohttp.
- **Ограниченная производительность** — при большом объёме данных или множестве одновременных соединений уступает специализированным решениям.
- **Дополнительные зависимости** — для отдельных задач (например, WebSockets) нужны сторонние библиотеки.
- **Не всегда хватает возможностей** — для продвинутых сценариев иногда нужны более специализированные инструменты.

Вывод: для большинства задач парсинга возможностей Requests более чем достаточно.



Метод requests.get()

`requests.get()` используется для чтения данных, не изменяя их на сервере.

```
import requests
response = requests.get(url='http://httpbin.org/')
```

Параметр	Назначение
<code>url</code>	адрес запроса (обязательный)
<code>params</code>	словарь параметров для строки запроса
<code>headers</code>	словарь HTTP-заголовков
<code>cookies</code>	куки, отправляемые с запросом
<code>auth</code>	данные для базовой HTTP-аутентификации
<code>timeout</code>	максимальное время ожидания ответа, сек.
<code>allow_redirects</code>	разрешить ли автоматические перенаправления
<code>proxies</code>	словарь прокси-серверов для запроса



`Response` инкапсулирует всё, что вернул сервер: статус-код, заголовки, содержимое ответа и время выполнения запроса — и предоставляет доступ к ним через атрибуты.

```
response = requests.get('https://httpbin.org/get')
```

Обращение к атрибутам —
через точку: `response.status_code`

```
Response {
  status_code: 200,
  headers: {...},
  url: 'https://httpbin.org/get',
  text: '...',
  content: b'...',
  json: <method>,
  elapsed: <datetime.timedelta>,
  ...
}
```



Атрибуты объекта Response

7

Атрибут	Что возвращает
<code>status_code</code>	HTTP-код статуса ответа
<code>text</code>	содержимое ответа строкой
<code>content</code>	содержимое ответа в байтах
<code>json()</code>	десериализованный JSON-ответ
<code>headers</code>	заголовки, возвращённые сервером
<code>url</code>	итоговый URL запроса

Атрибут	Что возвращает
<code>encoding</code>	кодировка ответа
<code>elapsed</code>	время выполнения запроса
<code>cookies</code>	куки, возвращённые сервером
<code>history</code>	список перенаправлений
<code>ok</code>	True, если код ответа 2xx
<code>reason</code>	текстовое пояснение статуса («ОК» и т. п.)



Атрибуты Response: пример

8

```
import requests

url = "https://httpbin.org/user-agent"
response = requests.get(url)

print(response.status_code)
# 200

print(response.text)
# '{"user-agent": ...}'

json_data = response.json()
# {'user-agent': ...}

print(response.headers)
```

```
print(response.url)
# 'https://httpbin.org/user-agent'

print(response.encoding)
# 'utf-8'

print(response.elapsed)
# 0:00:00.312000

print(response.ok)
# True

print(response.reason)
# 'OK'
```



`response.text`

Тип данных:

Обычная строка Python.

Обработка:

Не обрабатывает данные — при необходимости строку нужно парсить самостоятельно.

Когда использовать:

HTML, XML, текстовые ответы.

`response.json()`

Тип данных:

Словарь или список.

Обработка:

Автоматически декодирует JSON; если строка не валидный JSON — ошибка `JSONDecodeError`.

Когда использовать:

Ответы API в формате JSON.

`response.content`

Тип данных:

Байтовая строка.

Обработка:

«Сырые» байты без какой-либо обработки.

Когда использовать:

Картинки, аудио, видео и другие бинарные файлы.

Кодировку ответа при необходимости можно поменять с помощью `response.encoding`.



`headers` — словарь, который передаётся в `.get()` и определяет, как сервер воспримет ваш запрос: под каким браузером, с какими настройками и куки.

Тип заголовка	Описание	Примеры
Общие	Есть и в запросах, и в ответах, но не относятся к телу сообщения	<code>Cache-Control</code> , <code>Connection</code>
Заголовки запроса	Отправляются только в запросе: информация о клиенте	<code>Accept</code> , <code>User-Agent</code>
Заголовки ответа	Отправляются только в ответе: информация о сервере	<code>Server</code> , <code>WWW-Authenticate</code>
Сущностные	Описывают тело сообщения, его тип и модификации	<code>Content-Type</code> , <code>Content-Encoding</code>

При парсинге рекомендуется отправлять собственные заголовки вместе с запросом — сервер решает, что делать с запросом, во многом ориентируясь именно на них.



Многие сайты защищаются от ботов и веб-скраперов, анализируя HTTP-заголовки. Стандартный User-Agent библиотеки Requests явно выдаёт, что запрос отправлен скриптом, а не браузером:

```
{  
    "user-agent": "python-requests/2.28.2"  
}
```

Решение — подставить заголовки настоящего браузера:

```
headers = {  
    'User-Agent': 'Mozilla/5.0',    # тип браузера  
    'Accept': 'text/html,application/xhtml+xml', # типы контента, которые клиент может обработать  
    'Connection': 'keep-alive'      # указание на использование постоянного соединения  
}  
  
response = requests.get(url, headers=headers)
```



Как снизить риск блокировки

12

1

Смена User-Agent

Использовать список разных User-Agent и выбирать случайный при каждом запросе.

2

IP-ротация

Использовать прокси-серверы для смены IP-адреса, если сервер блокирует по IP.

3

Задержки между запросами

`time.sleep()` между запросами снижает вероятность блокировки.

4

Обработка капчи

В некоторых случаях требуется распознавание и решение капчи.

5

Реферер и другие заголовки

Настройка дополнительных HTTP-заголовков для имитации поведения реального пользователя.

6

Сессии и куки

Использование сессий и сохранение куки делает скрапинг менее подозрительным.

7

Анализ JavaScript

Если сайт требует выполнения JS для доступа к контенту — нужен инструмент вроде Selenium.



`timeout` задаёт максимальное время (в секундах), которое `.get()` будет ждать ответа сервера. По умолчанию Requests ждёт неопределённо долго — почти всегда стоит задавать тайм-аут явно.

`timeout=1.5` — ждать 1,5 секунды. `timeout=(3.05, 27)` — отдельно тайм-аут подключения и тайм-аут чтения.

```
import requests
from requests.exceptions import ConnectTimeout

try:
    response = requests.get("http://example.com", timeout=5)
except ConnectTimeout:
    print("Сервер не ответил вовремя")
```

Совет: для «быстрых» запросов ставьте короткий тайм-аут, а для медленных (когда ожидается большой объём данных) — увеличивайте его.



Параметры запроса: params

14

`params` — словарь Python, который Requests сам конвертирует в строку запроса и добавляет к URL после знака `?`, формируя таким образом полный URL

```
params = {'key1': 'value1', 'key2': 'value2'}

r = requests.get('https://httpbin.org/get', params=params)
print(r.url)

>>> https://httpbin.org/get?key1=value1&key2=value2
```

Для отправки данных на сервер (например, создания записи) используют POST с телом запроса:

```
data = {'name': 'John', 'age': 30}
response = requests.post('https://api.example.com', json=data)
```



Когда использовать params

15

1 **Работа с API** — большинство API требуют передачи параметров для доступа к нужным данным.

2 **Динамические сайты** — контент страницы зависит от значений в URL.

```
https://example.com/search?query=note&brand=Dell&sort=price_asc
```

3 **Пагинация** — параметр страницы указывает, с какой страницы собирать данные.

```
https://example.com/news?page=2
```

4 **Фильтрация данных** — например, список товаров после определённого года и жанра.

```
https://api.example.com/movies?year_after=2000&genre=action
```



Requests позволяет отправлять запросы через прокси-сервер, скрывая реальный IP-адрес — это снижает риск бана при большом количестве запросов.

Протокол HTTP

Не шифрует передаваемые данные — уязвим для перехвата и модификации трафика.

Протокол HTTPS (HTTP Secure)

Использует SSL/TLS-шифрование; сайты чаще применяют строгие анти-скрапинг меры и CAPTCHA.

```
proxies = {'http': 'http://10.10.1.10:3128', 'https': 'http://10.10.1.10:1080'}  
response = requests.get(url, proxies=proxies)
```



Диапазон	Категория
100–199 / 200–299 / 300–399	Информационные / Успешные / Перенаправления

Код	Значение для парсинга
200	Успешно — ответ пришёл полностью и без ограничений
301 / 302	Ресурс переехал — парсер переходит по новому адресу
400 / 403 / 404	Неверный запрос / доступ запрещён / ресурс не найден
429	Слишком много запросов — превышен лимит, нужно подождать
500 / 502 / 503	Ошибка на стороне сервера — от парсера не зависит

`response.status_code` — в 95% случаев парсеру важен именно код **200**: он гарантирует, что данные получены полностью.



Сетевая работа всегда подвержена сбоям — тайм-аутам, обрывам соединения, ошибкам сервера. Умение перехватывать исключения помогает создавать устойчивые и надёжные парсеры, помогая разработчикам быстро идентифицировать проблемы в случае их возникновения.

```
try:
    r = requests.get(url)
except Exception as e:
    print(f"Ошибка: {e}")
```

Общее исключение — «широкая сеть», ловит любую ошибку, но не даёт понять, что именно пошло не так.

```
try:
    r = requests.get(url)
except requests.ConnectionError as e:
    print(f"Ошибка соединения: {e}")
```

Конкретное исключение — «мелкая сеть», ловит определённый тип ошибки и даёт точную информацию о проблеме.



Исключение	Когда возникает
<code>RequestException</code>	Базовое исключение — перехватывает любую ошибку библиотеки
<code>ConnectionError</code>	Не удалось подключиться: нет интернета или сервер недоступен
<code>HTTPError</code>	Сервер вернул код ошибки, отличный от 2xx
<code>MissingSchema</code>	URL некорректен или отсутствует (например, без http://)
<code>TooManyRedirects</code>	Слишком много перенаправлений подряд
<code>ConnectTimeout</code>	Сервер не ответил за отведённое время подключения
<code>ReadTimeout</code>	Сервер не прислал данные за отведённое время чтения
<code>Timeout</code>	Общее исключение для <code>ConnectTimeout</code> и <code>ReadTimeout</code> вместе
<code>JSONDecodeError</code>	Ответ сервера не удалось декодировать как JSON



```
import requests

r = requests.get(
    url,
    params=params,
    headers=headers,
    proxies=proxies,
    timeout=5,
)

r.status_code    # код ответа
r.text / r.json() # содержимое
```

Перед парсингом сайта:

- задайте свои headers (User-Agent);
- указывайте timeout на каждый запрос;
- проверяйте status_code перед обработкой;
- оборачивайте запрос в try/except;
- при частых запросах — паузы и/или прокси.

Дальше: получив HTML-код страницы через Requests, переходим к его разбору с помощью BeautifulSoup.